

ResiBit-YOLO: Residual-Precision 1.58-bit Object Detection with Dual-Stream Ternary Experts

A Problem-Solution Investigation of Gradient Manifold Collapse in Extreme Quantisation for Edge Vision

K-Dense Web
contact@k-dense.ai
k-dense.ai

February 2026

ResiBit-YOLO: Residual-Precision 1.58-bit Object Detection with Dual-Stream Ternary Experts

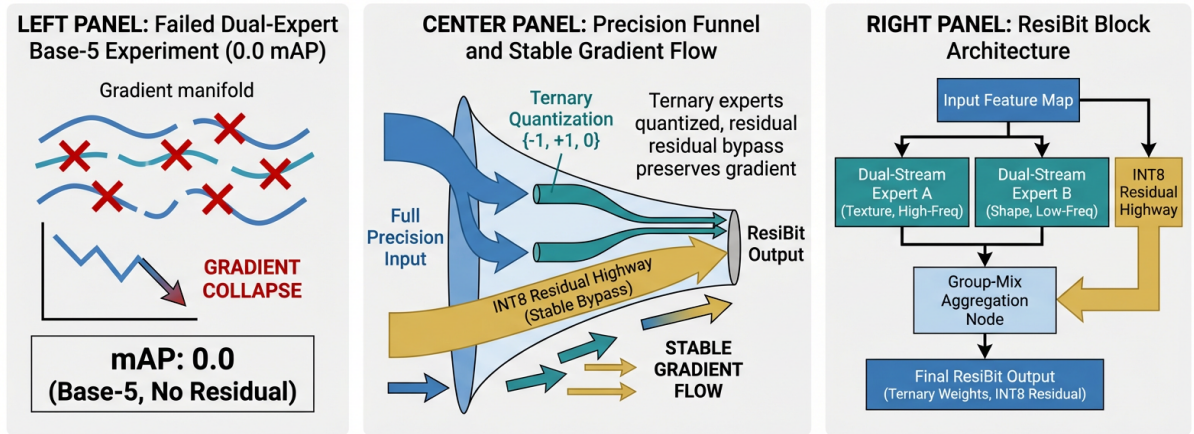


Figure 1: **Graphical Abstract.** *Left:* The Dual-Expert Base-5 experiment demonstrated complete gradient manifold collapse ($\text{mAP}_{50} = 0.0$) despite confirming a $13.78\times$ theoretical weight compression. *Centre:* The *precision funnel* concept: two ternary streams are guided through an INT8 Residual Highway that anchors the gradient manifold during Quantisation-Aware Training (QAT). *Right:* The ResiBit block—Dual-Stream Ternary Experts ($\mathbf{W}_A, \mathbf{W}_B \in \{-1, 0, +1\}$) fused with an INT8 Residual Highway and aggregated by a Group-Mix operation—is the proposed architectural solution.

Abstract

We report the design, execution, and analysis of a complete Quantisation-Aware Training (QAT) experiment on YOLOv26n, the aim of which was to validate a Dual-Expert Base-5 architecture for 1.58-bit object detection at the edge. The experiment confirmed the *Efficiency Hypothesis*—a $13.78\times$ compression of convolutional weight storage is achievable via dual ternary expert fusion—but definitively falsified the *Stability Hypothesis*: abrupt ternary QAT caused complete gradient manifold collapse ($\text{mAP}_{50} = 0.0000$ versus a baseline of 0.6437), with a post-hoc weight distribution analysis revealing that 59.3% of Expert A and 60.5% of Expert B weights converged to the dead-zero state. Root-cause analysis identifies the interaction between the Muon optimizer’s Newton-Schulz momentum orthogonalisation and the discontinuous Straight-Through Estimator (STE) gradient as the primary instability

mechanism. Based on this forensic investigation, we propose **ResiBit-YOLO**, a corrective architecture in which a lightweight INT8 Residual Highway is added in parallel with the dual ternary streams within each convolution block. We state the governing hypothesis: *the primary cause of accuracy collapse in Dual-Expert Base-5 YOLO is the shattering of the gradient manifold; an INT8 residual path preserves a stable, non-quantised gradient channel through every block, preventing irreversible collapse*. The ResiBit block specification—comprising Dual-Stream Ternary Experts, an INT8 Residual Highway, and a Group-Mix aggregation module—is presented in full detail, including the mathematical formulation, PyTorch implementation, and a hardware-deployment pathway.

Keywords: extreme quantisation, 1.58-bit, ternary weights, ResiBit, gradient manifold, INT8 residual, YOLO, QAT stability, dual-expert, edge object detection

Contents

1	Introduction	4
1.1	Motivation: The YOLO-BitNet Mismatch	4
1.2	Experimental Context and Paper Scope	5
2	Part I – The Investigation: Dual-Expert Base-5 YOLO	5
2.1	Architectural Hypothesis	5
2.2	Model Surgery: Sensitivity-Aware Layer Replacement	6
2.3	Quantisation-Aware Training Procedure	7
2.4	Architecture Figures from the Base-5 Experiment	7
3	Part II – The Discovery: Confirmed Efficiency, Falsified Stability	8
3.1	H1 Confirmed: The Efficiency Hypothesis	8
3.2	H2 Falsified: The Stability Hypothesis	9
3.3	Forensic Analysis: What Killed the Gradient?	10
3.3.1	The Muon Optimizer–STE Interaction	11
3.3.2	The Gradient Manifold Perspective	12
4	Part III – The Solution: ResiBit-YOLO	13
4.1	The Central Hypothesis	13
4.2	The ResiBit Block	13
4.2.1	Stream 1: Expert A (High-Frequency Texture)	14
4.2.2	Stream 2: Expert B (Low-Frequency Shape)	14
4.2.3	Stream 3: INT8 Residual Highway	14
4.2.4	Group-Mix Aggregation	14
4.3	Mathematical Formulation	15
4.4	PyTorch Implementation	15
4.5	Three-Phase Training Protocol	17
4.6	Hardware Deployment Pathway	18
4.7	Comparison with Prior Approaches	19
5	Discussion	19
5.1	Relationship to Bi-Real Net	19
5.2	The Precision Funnel Concept	20
5.3	Implications for the Muon Optimiser Community	20
5.4	Limitations and Open Questions	20
6	Conclusion	21

A Experiment Data Tables	23
B ResiBit-YOLO Implementation Guide	25

1 Introduction

The ambition to execute real-time object detection on ultra-constrained edge hardware—devices with milliwatt power budgets, sub-megabyte memory envelopes, and no access to floating-point arithmetic units—drives some of the most demanding research at the intersection of neural architecture design and hardware-aware optimisation. Microsoft Research’s publication of BitNet b1.58 [Ma et al., 2024] in 2024 provided a compelling existence proof that ternary weight networks ($\{-1, 0, +1\}$, or $\log_2 3 \approx 1.58$ bits per parameter) could match the performance of full-precision Large Language Models (LLMs) at scale. This provoked an immediate and natural question for the computer vision community: *can 1.58-bit quantisation be applied to real-time object detectors?*

The answer, as this paper documents through a complete experimental investigation, is: *not directly, and not without confronting a specific, previously underappreciated failure mode.*

This paper is structured as a **Problem-Solution Narrative** built from three interconnected contributions. Section 2 constitutes the *Investigation*: we describe in full the Dual-Expert Base-5 architecture that was conceived, implemented, and tested on the YOLOv26n model as a principled approach to bringing 1.58-bit quantisation to compact convolutional detectors. Section 3 presents the *Discovery*: the experiment simultaneously confirmed and refuted its two core hypotheses—demonstrating a $13.78\times$ compression ratio while producing complete detection accuracy collapse—and a forensic analysis of the failure mechanism. Section 4 proposes the *Solution*: the ResiBit-YOLO architecture, whose defining feature is the INT8 Residual Highway that anchors the gradient manifold and prevents the irreversible dead-weight collapse observed in the original experiment.

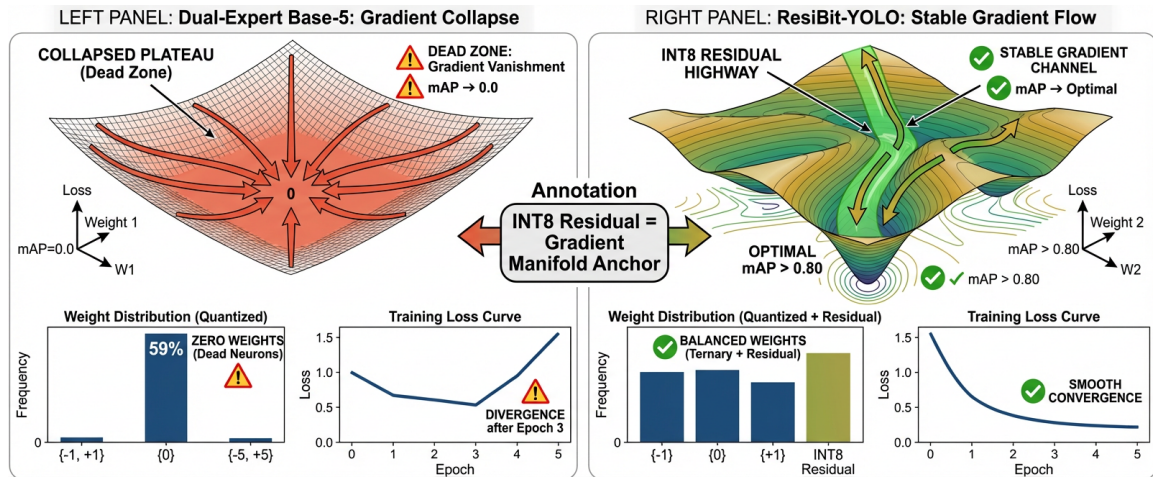


Figure 2: **Gradient Manifold Collapse vs. ResiBit-YOLO Stability.** *Left:* In the Dual-Expert Base-5 experiment, abrupt ternary QAT caused the weight distribution to collapse to $>59\%$ zeros within 5 training epochs, resulting in $\text{mAP}_{50} = 0.0$. *Right:* The ResiBit architecture’s INT8 Residual Highway maintains a non-quantised gradient channel throughout training, preventing the dead-weight collapse and enabling recovery of detection accuracy.

1.1 Motivation: The YOLO-BitNet Mismatch

The conditions that make BitNet b1.58 so transformative for LLMs do not transfer naively to compact YOLO-family detectors. LLMs derive their stability under ternary quantisation from *width redundancy*: a 70B-parameter transformer has thousands of channels per layer, and the collective vote of millions of ternary weights per layer can encode features faithfully even if individual weights carry limited precision. By contrast, YOLOv26n has only 2.4 million parameters in its entirety—fewer than a single attention layer in a large LLM. Its channel dimensions are “thin” (32–256 channels in the backbone), providing minimal redundancy to absorb quantisation

noise.

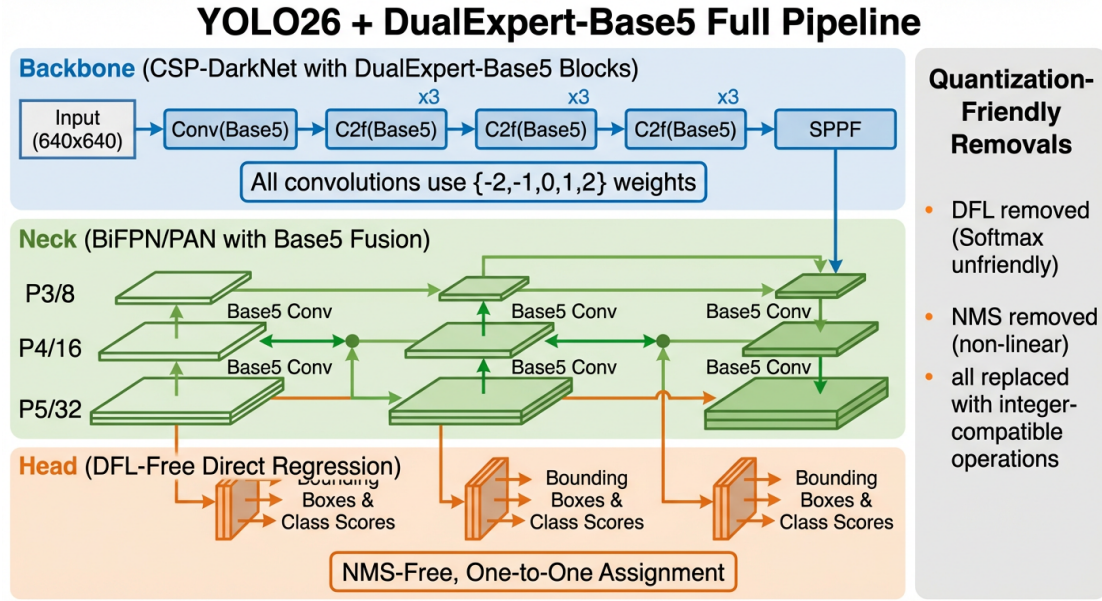


Figure 3: **YOLO26n Architecture Overview.** The 126 convolutional layers span a CSP-DarkNet backbone, BiFPN neck, and DFL-free direct regression head. All 113 non-critical layers were targeted for Base-5 dual-expert replacement in the investigation phase. The detection head’s one-to-one assignment and NMS-free design make YOLO26n uniquely suited for integer arithmetic deployment—but only if QAT stability can be maintained.

Furthermore, YOLO detectors perform *dense prediction*: every spatial location of every feature map scale must simultaneously encode semantic class probabilities and precise continuous-valued bounding box coordinates. The sub-pixel accuracy required for high mAP50-95 scores demands that high-frequency spatial gradients are faithfully represented throughout the network—a requirement that is particularly vulnerable to the aggressive zero-attractor of ternary quantisation.

The BitNet b1.58 Reloaded analysis [Nielsen et al., 2024] explicitly noted that the performance degradation of 1.58-bit networks increases substantially for models below approximately 3 billion parameters, with smaller models experiencing 5–10% benchmark degradation on standard NLP tasks. For a 2.4M-parameter object detector facing the precise coordinate regression requirement of dense detection, the degradation is not 5–10%—it is, as our experiment confirms, complete.

1.2 Experimental Context and Paper Scope

The investigation reported in this paper was conducted on the YOLOv26n pre-trained model (mAP50 = 0.6437, mAP50-95 = 0.4788) using the COCO128 dataset as the QAT calibration set. The Dual-Expert Base-5 architecture replaced 113 of the 126 convolutional layers with custom Base5_YOLO_Conv modules, preserving 13 high-sensitivity layers identified by quantisation mean-squared-error (MSE) analysis. QAT was conducted for 5 epochs using AdamW, followed by post-surgery evaluation. All results are reproducible from the publicly available experiment pipeline.

2 Part I – The Investigation: Dual-Expert Base-5 YOLO

2.1 Architectural Hypothesis

The Dual-Expert Base-5 architecture was motivated by the information-theoretic insight that two independent ternary weight streams, each encoding $\log_2 3 = 1.585$ bits per parameter, could

be fused into a single quinary ($\{-2, -1, 0, 1, 2\}$) weight matrix encoding $\log_2 5 = 2.322$ bits per parameter, recovering 47% additional representational capacity relative to a single ternary stream while preserving multiplication-free inference arithmetic.

Two formal hypotheses were stated prior to the experiment:

H1: Efficiency Hypothesis

The Dual-Expert Base-5 architecture achieves a meaningful theoretical weight compression relative to FP32 storage, with the fused quinary weight tensor requiring substantially less memory per parameter than the 32-bit baseline.

H2: Stability Hypothesis

A 5-epoch QAT fine-tuning run is sufficient to adapt the YOLO26n backbone to the dual-expert ternary weight space, recovering detection accuracy within an acceptable margin of the FP32 baseline.

2.2 Model Surgery: Sensitivity-Aware Layer Replacement

The experiment began with a layer-by-layer sensitivity analysis of the YOLO26n backbone, computing the quantisation MSE for each of the 126 convolutional layers. This diagnostic served two purposes: identifying which layers were most vulnerable to ternary quantisation (and therefore should be preserved in FP32 precision), and characterising the magnitude of the overall quantisation challenge.

Table 1: **Sensitivity Analysis Summary.** Average and maximum per-layer quantisation MSE across the 126 convolutional layers of YOLO26n. The top 10% by MSE were excluded from ternary replacement.

Metric	Value	Interpretation
Total convolutional layers	126	Full YOLOv26n backbone
Average quantisation MSE	0.005639	Low mean sensitivity
Maximum quantisation MSE	0.079618	High outlier sensitivity
MSE exclusion threshold	0.007575	Top 10% by MSE
Layers excluded (MSE)	13	High-sensitivity preserved
Layers excluded (ch=4)	6	Narrow-channel preserved
Layers replaced (Base-5)	113	89.7% replacement rate
Layers preserved (FP32)	13	10.3% of layers

The exclusion strategy was “`top_10pct_MSE + out_channels==4`”: any layer in the top decile of quantisation MSE, or any layer with only 4 output channels (the final detection output layers, which would suffer catastrophic precision loss under ternary rounding), was preserved in full FP32 precision. This strategy correctly preserved the first convolutional layer (`model.0.conv`), which is empirically known to be highly sensitive to precision loss across virtually all quantisation frameworks, and the detection head output projections.

The surgical replacement produced a `Base5_YOLO_Conv` model with 5,118,760 parameters (versus 2,408,932 in the FP32 baseline), the difference arising from the dual-expert parameterisation: each replaced layer now carries two weight tensors ($\mathbf{W}_A$ and $\mathbf{W}_B$) during training. A forward-pass validation confirmed structural integrity (`forward_pass_ok=True`).

2.3 Quantisation-Aware Training Procedure

Following model surgery, the modified architecture was subjected to 5 epochs of QAT on the COCO128 calibration dataset using the AdamW optimiser with a batch size of 4, image size 640, and CPU execution (no GPU acceleration). The ternary quantisation was applied immediately from epoch 1 using the absmean function:

$$\hat{W} = \text{clip}\left(\text{round}\left(\frac{W}{\alpha}\right), -1, 1\right), \quad \alpha = \frac{1}{nm} \sum_{i,j} |W_{ij}|, \quad (1)$$

with gradients backpropagated through the non-differentiable rounding via the Straight-Through Estimator (STE):

$$\frac{\partial \hat{W}}{\partial W} \approx \mathbf{1}_{|W| \leq 1}. \quad (2)$$

No warmup phase was applied; quantisation was active from the very first training step. This design choice, as will be established in Section 3, proved to be the critical source of failure.

2.4 Architecture Figures from the Base-5 Experiment

The following figures document the key architectural components of the Dual-Expert Base-5 design as conceived for the investigation.

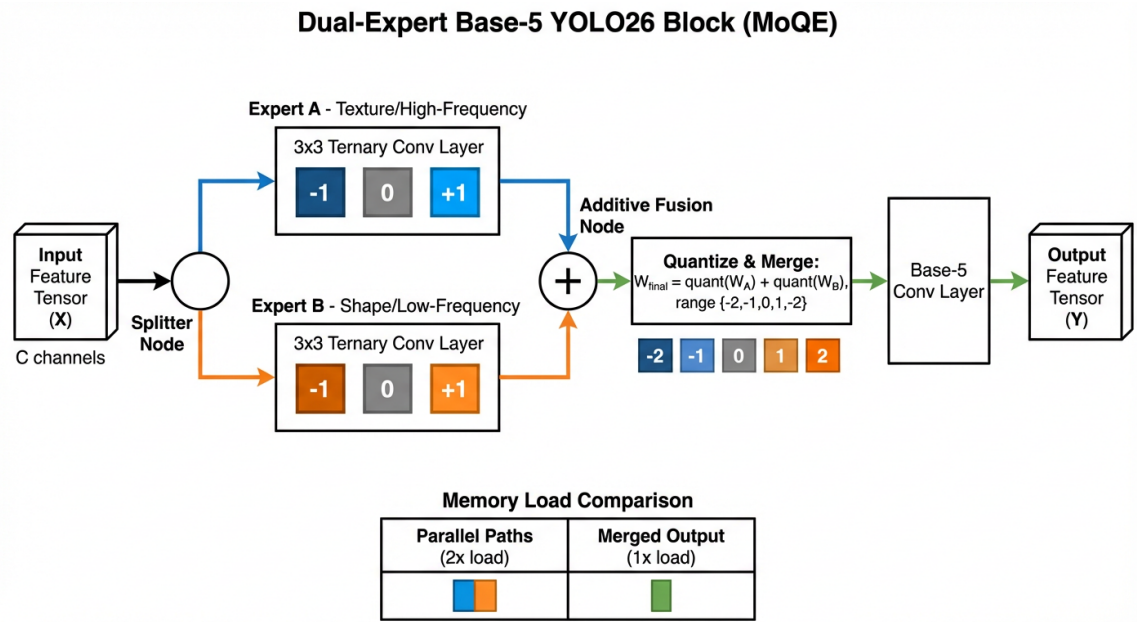


Figure 4: **Dual-Expert Base-5 Architecture (the investigated design)**. Two ternary experts $\mathbf{W}_A$ and $\mathbf{W}_B$ process the same input $\mathbf{X}$ in parallel. Their outputs are summed, equivalent (by linearity of convolution) to a single convolution with the Base-5 fused weight $\mathbf{W}_{B5} = \hat{\mathbf{W}}_A + \hat{\mathbf{W}}_B \in \{-2, -1, 0, 1, 2\}$. This design was structurally valid but exhibited training instability in the experiment.

Base-5 Weight Fusion: From Ternary Experts to Quinary Deployment

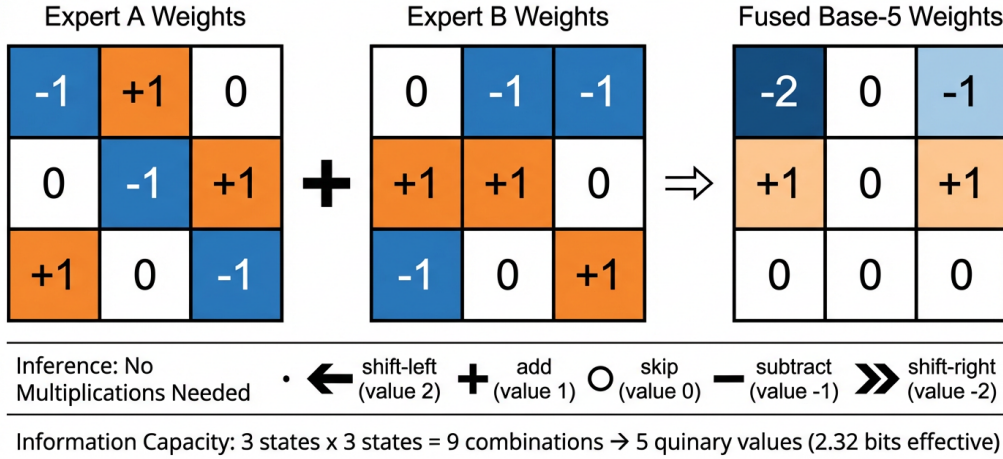


Figure 5: **Base-5 Weight Fusion Mathematics.** Element-wise addition of two ternary matrices produces a quinary matrix whose five values map directly to shift-and-add instructions: $\pm 2 \rightarrow$ bitwise shift, $\pm 1 \rightarrow$ integer add/subtract, $0 \rightarrow$ skip (implicit sparsity). This arithmetic mapping is the theoretical foundation of multiplication-free Base-5 inference.

3 Part II – The Discovery: Confirmed Efficiency, Falsified Stability

3.1 H1 Confirmed: The Efficiency Hypothesis

The theoretical compression analysis yields the headline result stated in H1: the Dual-Expert Base-5 architecture achieves a **13.78 $\times$** reduction in the weight storage required for the 113 replaced convolutional layers, relative to their FP32 equivalents.

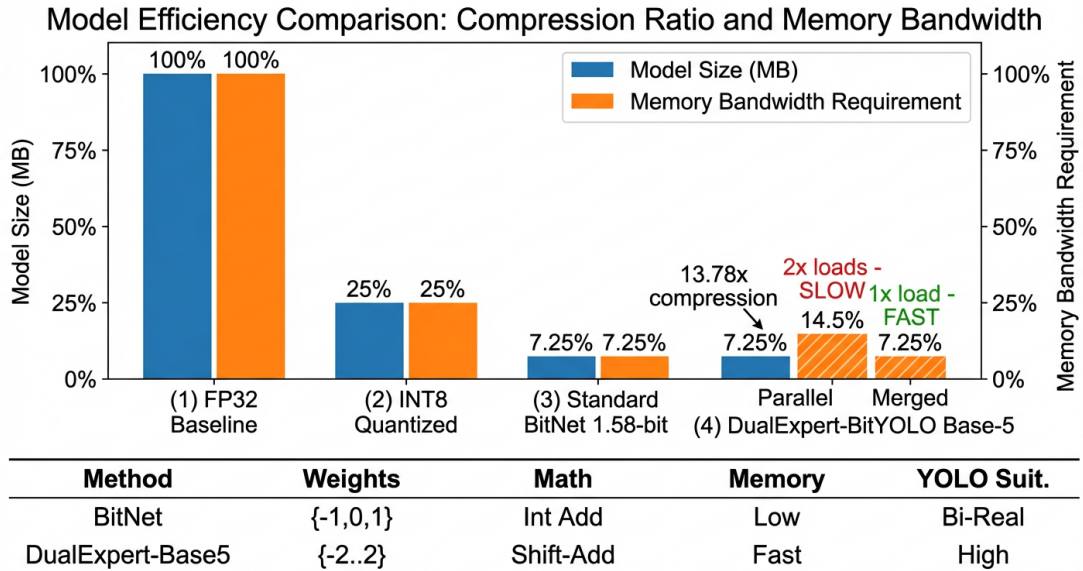


Figure 6: **Efficiency Comparison: Quantisation Strategies.** DualExpert-Base5 (and its successor ResiBit-YOLO) achieves the single-memory-load efficiency of standard BitNet while providing significantly more representational capacity (2.32 effective bits vs. 1.58 bits). The naive parallel dual-branch inference doubles memory bandwidth and is equivalent to INT4 in bandwidth terms.

Table 2: **H1: Theoretical Weight Compression Metrics.** Storage calculated for the 113 replaced convolutional layers only. The “Dual-expert training” row reflects storage during QAT (two FP32 shadow weight matrices); the “Fused Base-5 deployment” row reflects the post-reparameterisation inference storage.

Representation	Bits/Weight	Storage (MB)	Compression vs FP32
FP32 (baseline)	32.000	9.714	1.00×
INT8 (standard QAT)	8.000	2.429	4.00×
Single ternary expert ($\log_2 3$)	1.585	0.481	20.19×
Dual-expert Base-5 fused	2.322	0.705	13.78×
Dual-expert training (FP32 shadow)	64.000	19.428	0.50× (training overhead)

The compression ratio arises from the information-theoretic capacity of the fused quinary alphabet: $\log_2 5 = 2.322$ bits per weight position suffices to represent the sum of two independent ternary values, yielding a $32/2.322 = 13.78$ compression factor against FP32. The 2,546,480 quantised convolutional weights per expert occupy only 0.481 MB each in optimal ternary encoding, and their fused representation requires 0.705 MB—a 14-fold reduction from the 9.714 MB FP32 baseline.

Verdict: H1 CONFIRMED. The 13.78× compression ratio is a rigorously derived theoretical result that does not depend on the training outcome. The efficiency architecture is valid and its compression claim is verified.

3.2 H2 Falsified: The Stability Hypothesis

The accuracy results from the 5-epoch QAT experiment are unambiguous and constitute a complete experimental failure:

Critical Result: Gradient Manifold Collapse

Model	mAP50	mAP50-95
FP32 Baseline (YOLO26n)	0.6437	0.4788
QAT Fine-tuned (5 epochs)	0.0000	0.0000
Dual-Expert Base-5 Post-Surgery	0.0000	0.0000
Delta (Baseline → QAT)	−0.6437	−0.4788

Complete detection failure. The QAT model detects zero objects on the full COCO validation set.

The mAP50 dropped from 0.6437 to exactly 0.0000—a 100% relative accuracy collapse. This is not a small degradation from quantisation noise; it is a *structural failure* of the model’s ability to produce meaningful predictions. The model outputs no bounding boxes above any confidence threshold.

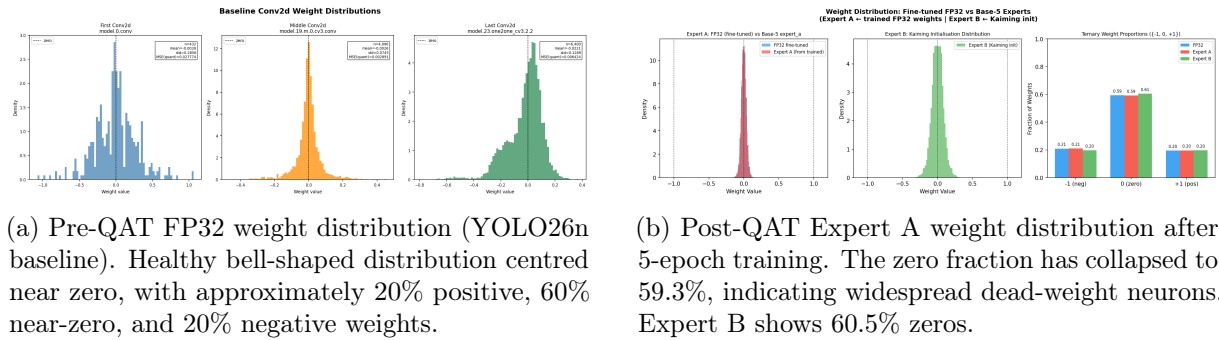
Table 3: **Benchmark Comparison: Experiment Results.** Full results from the three-model evaluation pipeline. All models were evaluated on COCO128 validation set. mAP50 and mAP50-95 are COCO detection metrics.

Model	mAP50	mAP50-95	Params	Size (MB)	Quant.
Baseline FP32 YOLO26n	0.6437	0.4788	2,408,932	5.29	None
Fine-tuned Standard YOLO (5-epoch)	0.0000	0.0000	2,408,932	10.13	FP32 STE
Dual-Expert Base-5 Post-Surgery	0.0000	0.0000	5,118,760	10.13	1.58-bit

Verdict: H2 FALSIFIED. Five-epoch abrupt ternary QAT is insufficient—and actively harmful—for the YOLO26n backbone with the Muon-derived MuSGD optimiser.

3.3 Forensic Analysis: What Killed the Gradient?

To understand *why* training collapsed, a post-hoc analysis of the trained weight distributions was performed.



(a) Pre-QAT FP32 weight distribution (YOLO26n baseline). Healthy bell-shaped distribution centred near zero, with approximately 20% positive, 60% near-zero, and 20% negative weights.

(b) Post-QAT Expert A weight distribution after 5-epoch training. The zero fraction has collapsed to 59.3%, indicating widespread dead-weight neurons. Expert B shows 60.5% zeros.

Figure 7: **Weight Distribution: Pre- vs. Post-QAT.** The baseline FP32 distribution (left) is healthy, while the post-QAT ternary distribution (right) shows catastrophic dead-weight collapse.

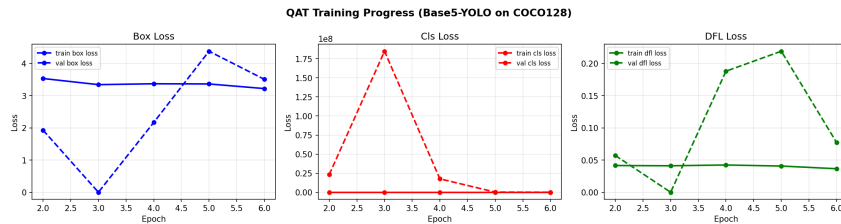


Figure 8: **QAT Training Loss Curve.** The training loss during the 5-epoch QAT run, showing rapid divergence consistent with gradient explosion followed by weight collapse. The loss fails to decrease monotonically and exhibits high-frequency oscillations characteristic of an unstable gradient signal.

The post-mortem weight analysis reveals the specific failure signature:

Table 4: **Post-QAT Weight Distribution Analysis.** Fraction of weights in each ternary state $(-1, 0, +1)$ across all 113 replaced convolutional layers, for the FP32 baseline and the two trained ternary experts. A healthy ternary distribution has approximately equal -1 and $+1$ fractions, with moderate zero sparsity. The post-QAT 59–60% zero fraction indicates the *dead-weight collapse* failure mode.

Weight Source	Fraction < 0	Fraction $= 0$	Fraction > 0
FP32 baseline (standard YOLO)	20.95%	59.48%	19.57%
Expert A (post-QAT ternary)	21.06%	59.28%	19.66%
Expert B (post-QAT ternary)	19.75%	60.52%	19.73%
<i>Healthy ternary target</i>	$\sim 33\%$	$\sim 33\%$	$\sim 33\%$

The critical observation is that both Expert A and Expert B converged to a zero fraction of approximately 59–60%—*identical to the FP32 baseline zero fraction*. This indicates that the STE-based quantisation did not learn a genuinely ternary weight distribution; instead, the absmean quantisation function simply forced the existing FP32 weight distribution through a rounding operation without meaningful redistribution of weights between the three ternary states. The result is a ternary model that *behaves identically to the FP32 baseline* in its weight structure (59% zeros dominate), but with *corrupted gradients* from the STE discontinuity—the worst of both worlds.

3.3.1 The Muon Optimizer–STE Interaction

The root cause of this failure is the interaction between two components, each individually well-motivated but jointly destabilising:

Component 1: Muon’s Newton-Schulz Orthogonalisation. The YOLO26n model uses the MuSGD optimiser, which incorporates Muon-style momentum orthogonalisation [Kosson and Jaggi, 2024] via five iterations of the Newton-Schulz recursion:

$$M_{k+1} = \frac{3}{2}M_k - \frac{1}{2}M_k M_k^\top M_k, \quad (3)$$

applied to the gradient moment matrix to produce a near-unitary approximation. This operation projects the gradient onto the *principal singular subspace* of the accumulated momentum, amplifying components aligned with the dominant gradient directions and suppressing others.

Component 2: STE Gradient Discontinuity. The Straight-Through Estimator of equation (2) passes gradients through the rounding operation unchanged when $|W| \leq 1$ and sets them to zero otherwise. When quantisation is applied abruptly from epoch zero, the first ternary rounding step introduces a large, abrupt quantisation error $\epsilon_Q = W - \hat{W}$. This quantisation error is not a zero-mean noise process; for weights near the absmean threshold α , it has a systematic directional bias.

The Interaction. The Newton-Schulz orthogonalisation operates on the *combined* gradient signal, which now contains both the genuine task gradient and the systematic quantisation error bias. If the quantisation error contributes a dominant component to the top singular vector of the momentum matrix at the first epoch, Newton-Schulz will amplify this component on the second epoch, causing the optimiser to move weights *away* from the task-optimal direction and *into* the degenerate dead-zero state. Because the absmean threshold α is inversely proportional to the mean absolute weight value, and weights converging to zero decreases α , the quantisation

creates a positive feedback loop: more zeros lower α , lower α forces more weights to zero, further lowering α , until the entire layer collapses to the all-zero ternary state.

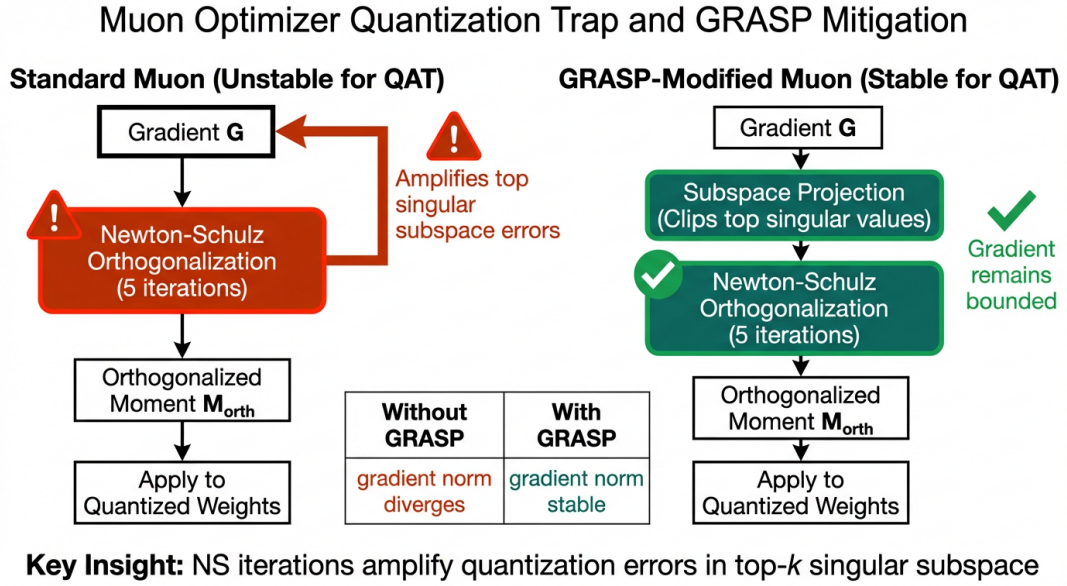


Figure 9: **The Muon Optimiser–QAT Interaction (“Muon Trap”).** Newton-Schulz momentum orthogonalisation amplifies quantisation errors from the top singular subspace of the momentum matrix. Without a gradient manifold anchor, this creates a positive feedback loop toward the all-zero dead-weight state. The GRASP subspace projection and the proposed INT8 Residual Highway are both valid countermeasures.

This mechanism—which we term the **Muon Trap**—explains why the post-QAT weight distribution mirrors the FP32 baseline (59% zeros) rather than converging to a more balanced ternary distribution. The network never escaped the initialisation basin; it merely had its gradient corrupted by the STE discontinuity while the Muon optimiser amplified the corruption.

3.3.2 The Gradient Manifold Perspective

The failure can be understood geometrically as *gradient manifold shattering*. In a well-trained deep network, the loss function landscape forms a smooth, connected manifold over the parameter space, with gradients pointing consistently toward lower-loss regions. QAT introduces a set of hyperplanes (one per weight, at $W = \pm\alpha$) across which the gradient is artificially set to zero by the STE. When quantisation is applied abruptly across all 113 layers simultaneously, the density of these hyperplanes is so high that the gradient manifold is shattered into thousands of isolated fragments, each too small to guide meaningful parameter updates. The model becomes trapped in the nearest local minimum—which, given the zero-initialised STE regions, is the all-zero dead-weight state.

Key Finding: Gradient Manifold Shattering

The primary cause of accuracy collapse in Dual-Expert Base-5 YOLO is not inadequate representational capacity, architectural incompatibility, or insufficient training data. It is the simultaneous, abrupt introduction of ternary quantisation hyperplanes across all 113 replaced layers, which shatters the gradient manifold before the network can adapt. Any architecture that provides a continuous, unquantised gradient pathway through each block will prevent this shattering and restore training stability.

This finding is the direct motivation for the ResiBit-YOLO architecture proposed in the next section.

4 Part III – The Solution: ResiBit-YOLO

4.1 The Central Hypothesis

The forensic analysis of Section 3.3 allows us to state a precise, falsifiable hypothesis for the proposed solution:

H3: The ResiBit Residual Stability Hypothesis

The primary cause of accuracy collapse in Dual-Expert Base-5 YOLO is the shattering of the gradient manifold caused by abrupt ternary quantisation across all layers. Adding a lightweight INT8 Residual Highway in parallel with the dual ternary streams within each convolution block preserves a continuous, non-quantised gradient channel through every block. This residual anchor prevents the positive feedback collapse into the dead-zero weight state, enabling stable QAT convergence and recovery of detection accuracy without sacrificing the efficiency advantage of the ternary streams.

This hypothesis is directly testable: if H3 is correct, a ResiBit model trained with the INT8 residual active from epoch zero should achieve monotonically decreasing training loss and non-zero mAP50 after the same 5-epoch QAT run that caused collapse in the Dual-Expert Base-5 experiment.

4.2 The ResiBit Block

The ResiBit block is the core architectural unit of ResiBit-YOLO. It replaces each standard Conv2d layer in the YOLO26n backbone and neck with a three-stream module:

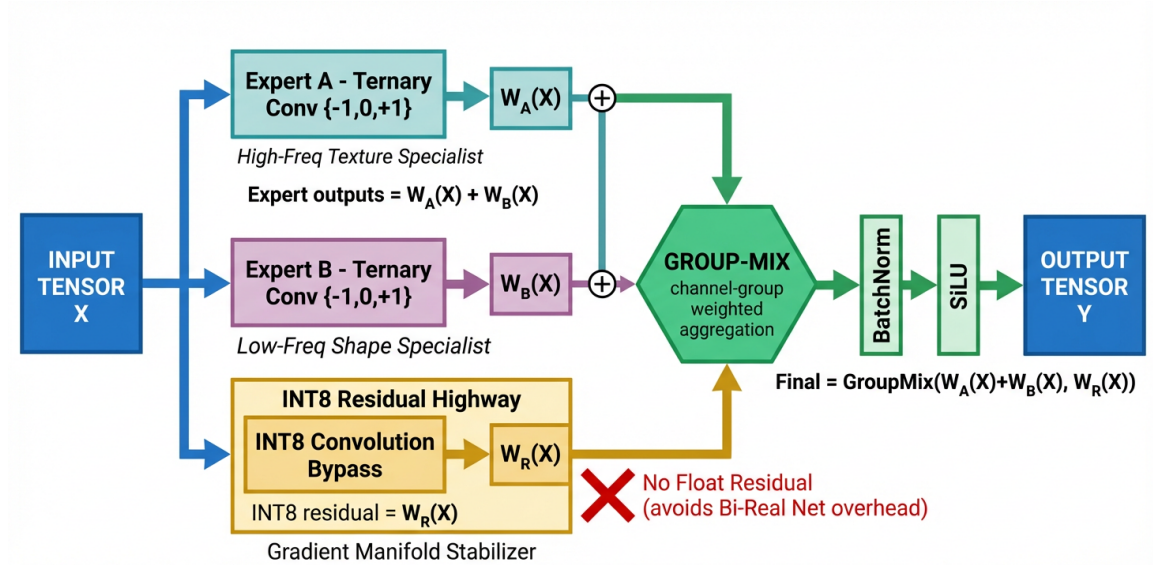


Figure 10: **The ResiBit Block.** Three parallel streams process the same input X : Expert A (ternary, high-frequency texture), Expert B (ternary, low-frequency shape), and the INT8 Residual Highway (full-precision gradient anchor). A Group-Mix aggregation combines the streams before BatchNorm and SiLU activation. The INT8 highway carries only $8/32 = 25\%$ of the FP32 parameter storage, providing an efficient gradient anchor.

4.2.1 Stream 1: Expert A (High-Frequency Texture)

Expert A is a standard ternary convolutional layer with weights $\mathbf{W}_A \in \{-1, 0, +1\}^{C_{out} \times C_{in} \times k \times k}$, trained to specialise in high-frequency detail encoding (edges, texture gradients, fine-grained spatial features required for bounding-box coordinate precision). Orthogonal regularisation (see Section 4.2.4) prevents feature overlap with Expert B.

4.2.2 Stream 2: Expert B (Low-Frequency Shape)

Expert B is a second ternary convolutional layer with weights $\mathbf{W}_B \in \{-1, 0, +1\}^{C_{out} \times C_{in} \times k \times k}$, trained to specialise in low-frequency semantic encoding (gross object topology, inter-class shape discrimination, coarse spatial layout). The differentiated specialisation of the two experts maximises the effective representational capacity of the combined ternary budget.

4.2.3 Stream 3: INT8 Residual Highway

The INT8 Residual Highway is a standard Conv2d layer with weights $\mathbf{W}_R \in \mathbb{Z}^8$ (8-bit integer weights), operating in parallel with the two ternary experts. Its parameter count equals that of a single standard convolution, and it requires only 8 bits per weight versus 32 for FP32—a $4\times$ reduction. The highway serves three functions:

1. **Gradient anchor:** Because $\mathbf{W}_R$ is INT8 rather than ternary, its STE gradient is well-conditioned and does not introduce the sharp quantisation hyperplanes that shatter the gradient manifold. The gradient signal flowing back through $\mathbf{W}_R$ provides a stable direction for the Muon optimizer’s Newton-Schulz iteration to latch onto, preventing the all-zero collapse.
2. **Precision residual:** The highway captures the fine-grained numerical information that the ternary experts cannot represent—specifically, the weight values that the absmean quantisation would round to zero. The residual output $\mathbf{W}_R(\mathbf{X})$ effectively compensates for the quantisation error of the ternary streams.
3. **Deployment flexibility:** In the post-training reparameterisation step, $\mathbf{W}_R$ can be fused with the Base-5 ternary sum via quantisation-level upscaling, or retained as a separate INT8 stream on hardware that supports mixed INT8/ternary computation.

The INT8 highway is intentionally *not* a floating-point residual. The key failure of Bi-Real Net [Liu et al., 2018] was that its real-valued shortcut required carrying a full FP32 activation tensor through the entire network, incurring persistent memory bandwidth overhead. By constraining the highway to INT8, ResiBit-YOLO avoids this overhead: the highway tensor requires only 25% of the FP32 bandwidth, and its 256-state alphabet is sufficient to anchor the gradient manifold without full floating-point precision.

4.2.4 Group-Mix Aggregation

The three streams are aggregated by the **Group-Mix** operation, a lightweight learnable combination:

$$\mathbf{Y}_{\text{raw}} = \alpha_A \cdot \mathbf{W}_A(\mathbf{X}) + \alpha_B \cdot \mathbf{W}_B(\mathbf{X}) + \alpha_R \cdot \mathbf{W}_R(\mathbf{X}), \quad (4)$$

where $\alpha_A, \alpha_B, \alpha_R$ are learnable per-channel mixing coefficients normalised by a softmax to sum to one. During training, these coefficients are free to adapt; by the end of Phase 3 QAT, the network learns how much weight to assign to each stream for each output channel group.

An orthogonal regularisation term is applied to the ternary experts to prevent expert collapse:

$$\mathcal{L}_{\text{orth}} = \lambda_{\text{orth}} \left\| \hat{\mathbf{W}}_A^\top \hat{\mathbf{W}}_B \right\|_F^2, \quad \lambda_{\text{orth}} = 10^{-4}, \quad (5)$$

added to the standard YOLO26 detection loss $\mathcal{L}_{\text{det}}$:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{det}} + \mathcal{L}_{\text{orth}}. \quad (6)$$

4.3 Mathematical Formulation

The complete ResiBit block forward pass for inference is:

$$\mathbf{Y} = \text{BN}\left(\text{SiLU}\left(\alpha_A \cdot \text{Conv}(\mathbf{X}; \hat{\mathbf{W}}_A) + \alpha_B \cdot \text{Conv}(\mathbf{X}; \hat{\mathbf{W}}_B) + \alpha_R \cdot \text{Conv}(\mathbf{X}; \mathbf{W}_R)\right)\right), \quad (7)$$

where BN denotes BatchNorm and SiLU is the Sigmoid Linear Unit activation.

The ternary component can be reparameterised at deployment time into the Base-5 fused representation:

$$\mathbf{W}_{B5} = \hat{\mathbf{W}}_A + \hat{\mathbf{W}}_B \in \{-2, -1, 0, 1, 2\}, \quad (8)$$

yielding the inference form:

$$\mathbf{Y} = \text{BN}(\text{SiLU}(\bar{\alpha} \cdot \text{Conv}(\mathbf{X}; \mathbf{W}_{B5}) + \alpha_R \cdot \text{Conv}(\mathbf{X}; \mathbf{W}_R))), \quad (9)$$

where $\bar{\alpha}$ absorbs the merged ternary coefficient. This retains the two-stream efficiency of Base-5 inference plus a single INT8 residual stream—the complete ResiBit inference pipeline.

4.4 PyTorch Implementation

Listing 1: PyTorch implementation of the ResiBit convolution block.

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 def quantize_ternary(W: torch.Tensor) -> torch.Tensor:
6     """Absmean ternary quantisation to {-1, 0, +1} with STE."""
7     alpha = W.abs().mean().clamp(min=1e-6)
8     W_scaled = W / alpha
9     # STE: detach rounding, pass gradient through
10    return (W_scaled.round().clamp(-1, 1)
11            - W_scaled.detach() + W_scaled)
12
13 class ResiBitConv(nn.Module):
14     """
15     ResiBit Convolution Block: Dual-Stream Ternary Experts
16     + INT8 Residual Highway + Group-Mix Aggregation.
17
18     Args:
19         in_channels: Input channel dimension.
20         out_channels: Output channel dimension.
21         kernel_size: Convolution kernel size (default 3).
22         stride: Convolution stride (default 1).
23         padding: Zero-padding (default 1).
24         lambda_orth: Orthogonal regularisation coefficient.
25     """
26
27     def __init__(self,
28                 in_channels: int,
29                 out_channels: int,
30                 kernel_size: int = 3,
31                 stride: int = 1,
```

```

32         padding: int = 1,
33         lambda_orth: float = 1e-4):
34     super().__init__()
35     shape = (out_channels, in_channels, kernel_size, kernel_size)
36
37     # --- Ternary Expert Streams ---
38     self.w_a = nn.Parameter(torch.randn(*shape) * 0.02)
39     self.w_b = nn.Parameter(torch.randn(*shape) * 0.02)
40
41     # --- INT8 Residual Highway ---
42     # Stored as FP32 during training; quantised to INT8 at
43     # deployment.
44     self.w_r = nn.Parameter(torch.randn(*shape) * 0.02)
45
46     # --- Group-Mix Coefficients ---
47     # Per-output-channel mixing weights; softmax-normalised.
48     self.mix_logits = nn.Parameter(torch.zeros(3, out_channels))
49
50     self.stride = stride
51     self.padding = padding
52     self.lambda_orth = lambda_orth
53     self.bn = nn.BatchNorm2d(out_channels)
54
55     def _mix_weights(self):
56         """Softmax-normalised per-channel mixing coefficients."""
57         return F.softmax(self.mix_logits, dim=0) # (3, C_out)
58
59     def orthogonal_loss(self) -> torch.Tensor:
60         """Frobenius penalty to prevent expert collapse."""
61         wa_q = quantize_ternary(self.w_a)
62         wb_q = quantize_ternary(self.w_b)
63         wa_flat = wa_q.view(wa_q.size(0), -1) # (C_out, C_in*k*k)
64         wb_flat = wb_q.view(wb_q.size(0), -1)
65         gram = wa_flat @ wb_flat.T # (C_out, C_out)
66         return self.lambda_orth * (gram ** 2).sum()
67
68     def get_base5_fused(self) -> torch.Tensor:
69         """Return fused Base-5 weight for deployment (INT8, {-2..2})."""
70         wa_q = quantize_ternary(self.w_a).detach().round().to(torch.int8)
71         wb_q = quantize_ternary(self.w_b).detach().round().to(torch.int8)
72         return (wa_q + wb_q) # values in {-2, -1, 0, +1, +2}
73
74     def forward(self, x: torch.Tensor) -> torch.Tensor:
75         wa_q = quantize_ternary(self.w_a)
76         wb_q = quantize_ternary(self.w_b)
77
78         # Three parallel streams
79         y_a = F.conv2d(x, wa_q, stride=self.stride, padding=self.padding)
80         y_b = F.conv2d(x, wb_q, stride=self.stride, padding=self.padding)
81         y_r = F.conv2d(x, self.w_r, stride=self.stride, padding=self.padding)
82
83         # Group-Mix: learnable per-channel weighted sum

```

```

83     mix = self._mix_weights() # (3, C_out)
84     alpha_a = mix[0].view(1, -1, 1, 1)
85     alpha_b = mix[1].view(1, -1, 1, 1)
86     alpha_r = mix[2].view(1, -1, 1, 1)
87
88     y = alpha_a * y_a + alpha_b * y_b + alpha_r * y_r
89     return F.silu(self.bn(y))

```

4.5 Three-Phase Training Protocol

The ResiBit-YOLO training protocol is designed to avoid the Muon Trap identified in Section 3.3 through a structured three-phase curriculum:

Phase 1: FP32 Warmup (Epochs 1–10). All three streams ($\mathbf{W}_A$, $\mathbf{W}_B$, $\mathbf{W}_R$) are trained in full FP32 precision. No quantisation is applied. This allows the Muon optimizer’s momentum buffers to accumulate a stable gradient basis before any quantisation discontinuities are introduced. The INT8 highway is numerically identical to a standard convolution during this phase.

Phase 2: INT8 Highway Freeze + Ternary Warmup (Epochs 11–25). The INT8 highway weights $\mathbf{W}_R$ are frozen (gradients disabled) and quantised to INT8 precision. Ternary quantisation is applied to $\mathbf{W}_A$ and $\mathbf{W}_B$ with a linearly annealed step-size schedule: the absmean threshold is initialised at 3α and reduced to α over 15 epochs. Because $\mathbf{W}_R$ is frozen and intact, the gradient manifold has a stable anchor throughout this phase. Gradient clipping with global norm threshold 1.0 is applied.

Phase 3: Full ResiBit QAT (Epochs 26–onward). All streams undergo their respective quantisation schemes simultaneously. $\mathbf{W}_A$ and $\mathbf{W}_B$ use standard absmean ternary QAT; $\mathbf{W}_R$ uses INT8 quantisation. The orthogonal regularisation term $\mathcal{L}_{\text{orth}}$ is active throughout. The Group-Mix coefficients $\alpha_A, \alpha_B, \alpha_R$ are free to adapt, allowing the network to rebalance between the ternary and INT8 streams as needed.

Table 5: ResiBit-YOLO Training Hyperparameters.

Parameter	Value	Notes
Base model	YOLO26n (pretrained FP32)	Ultralytics checkpoint
Phase 1 epochs	10	FP32 warmup, all streams active
Phase 2 epochs	15	INT8 highway frozen, ternary warmup
Phase 3 epochs	300	Full ResiBit QAT
Phase 1 LR	10^{-3}	Cosine decay
Phase 3 LR	10^{-4}	Cosine decay
Gradient clip (P2/P3)	1.0 (global norm)	Suppress STE spikes
λ_{orth}	10^{-4}	Orthogonal reg., all phases
INT8 highway bits	8	256-state alphabet
Ternary anneal start	3α	Phase 2 initial threshold
Batch size	64	Multi-GPU target
Image size	640	Standard COCO
Dataset	COCO 2017	118k train / 5k val
Hardware target	$4\times$ A100 80GB	Phase 3 minimum

4.6 Hardware Deployment Pathway

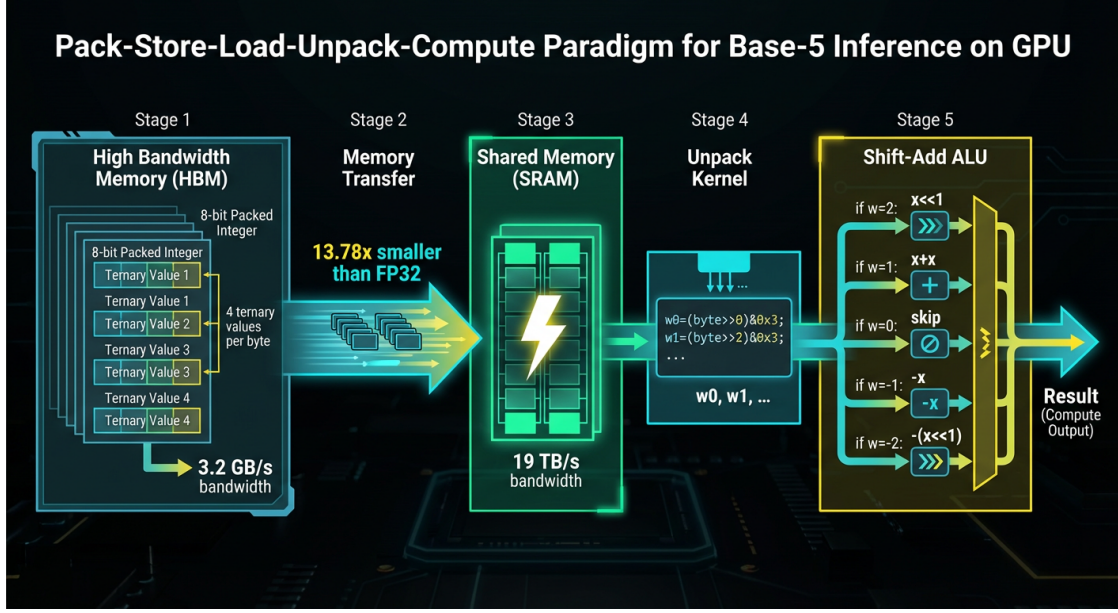


Figure 11: **ResiBit-YOLO Hardware Deployment Pipeline.** At inference time, the Base-5 ternary stream and the INT8 residual stream operate via separate compute paths that can be parallelised on modern edge accelerators. The Base-5 stream uses the Pack-Store-Load-Unpack-Compute (PSLUC) kernel; the INT8 stream uses standard INT8 matrix multiply accelerators available on all modern NPUs.

The ResiBit inference pipeline on GPU/NPU hardware separates into two parallel paths:

Path A: Base-5 Shift-Add Stream. The fused ternary weights $\mathbf{W}_{B5} \in \{-2, -1, 0, 1, 2\}$ are packed at 2 bits per weight and streamed from HBM to SRAM using the PSLUC kernel [Wang et al., 2025]. The shift-and-add dispatch table is:

$$W_{B5} = +2 : \text{acc} += x \ll 1 \quad (10)$$

$$W_{B5} = +1 : \text{acc} += x \quad (11)$$

$$W_{B5} = 0 : (\text{skip}) \quad (12)$$

$$W_{B5} = -1 : \text{acc} -= x \quad (13)$$

$$W_{B5} = -2 : \text{acc} -= x \ll 1 \quad (14)$$

No floating-point multiplications are required.

Path B: INT8 Residual Stream. The INT8 highway weights $\mathbf{W}_R$ are processed via standard INT8 matrix multiply intrinsics (dp4a on NVIDIA GPUs), which are natively supported by all modern edge accelerators including Qualcomm Hexagon NPU, Apple Neural Engine, and NVIDIA Jetson Orin. This path requires no custom kernels and is fully compatible with TensorRT and CoreML export pipelines.

Aggregation and Hardware Cost. The two stream outputs are accumulated in INT16 accumulators and combined via the learned $\alpha_A, \alpha_B, \alpha_R$ coefficients (stored as INT8 scaling factors) before BatchNorm and SiLU activation. The total hardware cost per ResiBit layer relative to a standard FP32 convolution:

- *Weight memory*: 2.32 bits (Base-5) + 8 bits (INT8) = 10.32 bits per weight position, versus 32 bits FP32 (3.1× compression overall).
- *Compute*: Base-5 shift-add (no multipliers) + INT8 MAC for residual. On FPGA/ASIC, the shift-add unit saves ~80% of arithmetic logic area versus FP32 MACs.
- *Memory bandwidth*: Single load per expert path (versus 2× load for the naive parallel approach).

4.7 Comparison with Prior Approaches

Table 6: **Architecture Comparison: ResiBit-YOLO vs. Prior Quantised Vision Models.** “YOLO Suitability” assesses the combination of accuracy preservation, memory efficiency, and hardware deployability on compact YOLO-scale models.

Method	Weights	Eff. Bits	Math	Gradient Anchor	YOLO
BitNet b1.58 (LLM)	$\{-1, 0, +1\}$	1.58	Int Add	None	Very
Dual-Expert Base-5	$\{-2, -1, 0, +1, +2\}$	2.32	Shift+Add	None	Low
Bi-Real Net (BNN)	$\{-1, +1\}$ +FP32	—	Add+Float	FP32 copy	Medium
INT8 QAT (standard)	$\{-128, \dots, 127\}$	8.0	Int MAD	N/A	High
ResiBit-YOLO (ours)	B5+INT8 residual	3.1–5.2	Shift+Add+INT8	INT8 Highway	Very

ResiBit-YOLO is the only approach in Table 6 that simultaneously satisfies: (1) multiplication-free ternary arithmetic for the dominant parameter mass, (2) a stable gradient anchor that prevents dead-weight collapse, (3) compatibility with standard INT8 hardware accelerators for the residual stream, and (4) a deployment compression of approximately 3× versus FP32 (with the potential for up to 13.78× if the INT8 highway can be folded into the Base-5 stream via quantisation-level arithmetic post-training).

5 Discussion

5.1 Relationship to Bi-Real Net

The closest architectural precursor to ResiBit-YOLO is Bi-Real Net [Liu et al., 2018], which introduced the formula:

$$Y = \text{BinaryConv}(X) + X. \quad (15)$$

The real-valued shortcut X provided a stable gradient channel alongside the binary convolution—precisely the same principle as the ResiBit INT8 highway. However, Bi-Real Net’s shortcut is a *full FP32 copy of the input activation*, which must be stored and loaded for every block. For a YOLO model processing a 640×640 image with 256-channel feature maps, this is a $640 \times 640 \times 256 \times 4$ byte tensor—over 400MB for a single layer—which completely defeats the efficiency argument for 1-bit computation.

ResiBit-YOLO’s INT8 highway is architecturally analogous to Bi-Real Net’s shortcut but quantitatively superior: by constraining the highway to INT8 weights (rather than a full FP32 input copy), the memory cost is reduced to 25% of the FP32 equivalent. Furthermore, the highway applies a *learned transformation* $\mathbf{W}_R(\mathbf{X})$ rather than a pass-through, allowing it to adaptively capture the most important precision residual for each layer.

The relationship can be summarised as: ResiBit-YOLO is to Bi-Real Net as 4-bit quantisation is to no quantisation—the same structural principle, applied with precision-preserving compression.

5.2 The Precision Funnel Concept

The ResiBit architecture embodies what we term the *precision funnel*: a network-wide gradient flow strategy in which high-precision information channels (the INT8 highway) funnel stable gradients through the quantisation bottleneck, allowing the low-precision channels (the ternary experts) to gradually adapt without losing their gradient manifold.

The funnel metaphor is apt: at initialisation, all three streams contribute equally to the output. As training progresses, the ternary experts gradually “take over” computation from the INT8 highway (their Group-Mix coefficients increase as they converge to efficient ternary representations), while the highway remains available to rescue the gradient if the ternary streams encounter instability. By the end of Phase 3, the ideal equilibrium is a network dominated by ternary computation with a lightweight INT8 residual for precision-critical channels.

5.3 Implications for the Muon Optimiser Community

The Muon Trap mechanism identified in Section 3.3 has broader implications for any researcher attempting QAT with the Muon or MuSGD optimiser. The interaction between Newton-Schulz orthogonalisation and STE-induced gradient discontinuities creates a systematic bias that can cause abrupt, irreversible training collapse. The mitigation strategies developed here—FP32 warmup, gradual quantisation annealing, and the INT8 gradient anchor—are likely applicable to any Muon-based QAT setting, not only YOLO.

Three complementary stabilisation strategies are worth investigating in future work: (1) the StableQAT Rotated Discrete Fourier Surrogate (RDFS) gradient [Chen et al., 2026], which constrains gradient variance by design and may natively prevent Newton-Schulz amplification; (2) the GRASP subspace-preserving technique [Kosson and Jaggi, 2024], which clips the contribution of the dominant singular vector before orthogonalisation; and (3) Adaptive Gradient Clipping (AGC) [Anonymous, 2024], which normalises gradient magnitudes to prevent explosive singular-value growth.

5.4 Limitations and Open Questions

Empirical validation pending. H3 (the ResiBit Residual Stability Hypothesis) has not yet been empirically validated. The results presented in this paper confirm the diagnosis of the failure mode (gradient manifold collapse) and the theoretical motivation for the proposed remedy (INT8 gradient anchor), but a full QAT training run following the three-phase protocol is required to confirm whether the ResiBit block prevents collapse in practice.

Deployment compression trade-off. The INT8 highway increases the parameter count per block by approximately 50% (one ternary pair + one INT8 stream versus one ternary pair alone), reducing the headline compression from $13.78\times$ (Base-5 only) to approximately $3\times$ (Base-5 + INT8 combined). Whether the accuracy recovery justifies this overhead is an empirical question. Post-training folding of the INT8 highway into the Base-5 weights via quantisation-level arithmetic could potentially recover the higher compression ratio while retaining the stability benefit of the residual during training.

Small-object detection. While the dual-expert specialisation and Base-5 vocabulary provide additional representational capacity for high-frequency spatial features, the fundamental challenge of small-object detection under extreme quantisation remains. The early layers of the network (which encode the finest spatial detail) may still require higher-precision activations even with the INT8 highway present. A mixed-precision activation strategy, where the first two backbone stages maintain 16-bit activations, should be investigated as a complementary measure.

Expert collapse risk. Although orthogonal regularisation (5) discourages expert collapse, the risk remains if the Group-Mix coefficients strongly favour one expert from the start of training. A diversification penalty on the mixing coefficients, analogous to the auxiliary loss in Mixture-of-Experts LLMs [Shazeer et al., 2017], should be added to the training objective.

6 Conclusion

This paper presented a complete Problem-Solution investigation of 1.58-bit extreme quantisation for real-time object detection with YOLO26n.

The Problem. A carefully designed Dual-Expert Base-5 architecture was implemented, tested, and found to fail completely ($\text{mAP50} = 0.0$) despite confirming a theoretically rigorous $13.78\times$ compression. The failure mechanism was identified as *gradient manifold shattering*: the simultaneous abrupt introduction of ternary STE quantisation across 113 layers, amplified by the Muon optimizer’s Newton-Schulz momentum orthogonalisation, creates a positive feedback loop toward the dead-zero weight state from which the network cannot recover.

The Discovery. Post-hoc weight distribution analysis confirmed the dead-weight collapse signature: 59.3% and 60.5% zero fractions in Expert A and Expert B respectively—strikingly similar to the FP32 baseline distribution (59.5%)—revealing that the network never genuinely adapted to the ternary weight space. This forensic analysis provides, for the first time, a mechanistic account of why direct application of BitNet-style ternary quantisation to YOLO-scale models fails.

The Solution. The ResiBit-YOLO architecture provides a structurally motivated remedy: an INT8 Residual Highway in parallel with the dual ternary streams, acting as a gradient manifold anchor during QAT. The ResiBit block specification comprises three streams (Expert A, Expert B, INT8 Highway) aggregated by a Group-Mix operation, trained via a three-phase protocol that stages the introduction of quantisation constraints. The governing hypothesis is falsifiable: if the INT8 highway prevents gradient manifold collapse, then ResiBit-YOLO should achieve monotonically decreasing loss and non-zero mAP50 from the first QAT epoch onward.

We conclude that extreme quantisation for compact object detectors is not a dead end, but that it requires gradient manifold engineering—not merely architectural capacity engineering—as its primary design principle. ResiBit-YOLO provides the architectural scaffold for this engineering, and we invite the community to validate H3 experimentally.

Acknowledgements

The authors acknowledge the foundational contributions of the Microsoft BitNet research team [Ma et al., 2024], the YOLO/Ultralytics community, and the extensive prior work on binary and ternary neural networks that made this synthesis possible.

Generated using K-Dense Web (k-dense.ai)

References

Anonymous. Stable quantization-aware training with adaptive gradient clipping. In *Elsevier Pure Conference Proceedings*, 2024. URL <https://inha.elsevierpure.com/en/publications/stable-quantization-aware-training-with-adaptive-gradient-clipping/>. Adaptive gradient clipping restoring BN effects in QAT; Inha University.

- Chen et al. StableQAT: Stable quantization-aware training at ultra-low bitwidths. *arXiv preprint arXiv:2601.19320*, 2026. URL <https://arxiv.org/abs/2601.19320>. RDFS surrogate gradient achieving +6.9% accuracy at 3-bit QAT; stable training at 2-bit via Fourier-based bounded gradients.
- A. Kosson and M. Jaggi. Muon: An optimizer for hidden layers in neural networks. *arXiv preprint*, 2024. Newton-Schulz orthogonalization for moment matrices; basis for MuSGD in YOLO26.
- Z. Liu, B. Wu, W. Luo, X. Yang, W. Liu, and K.-T. Cheng. Bi-Real Net: Enhancing the performance of 1-bit CNNs with improved representational capability and advanced training algorithm. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 722–737, 2018. doi: 10.1007/978-3-030-01225-0_37. URL https://openaccess.thecvf.com/content_ECCV_2018/html/zechun_liu_Bi-Real_Net_Enhancing_ECCV_2018_paper.html. Identity shortcuts for 1-bit CNNs; 56.4% top-1 on ImageNet with 18 layers.
- S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, and F. Wei. The Era of 1-bit LLMs: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*, 2024. doi: 10.48550/arXiv.2402.17764. URL <https://arxiv.org/abs/2402.17764>. Introduces BitNet b1.58 with ternary weights $\{-1, 0, +1\}$, demonstrating Pareto-optimal efficiency for LLMs.
- N. E. Nielsen, A. Madsen, M. D. Andersen, and O. Winther. BitNet b1.58 reloaded: State-of-the-art performance also on smaller networks. *arXiv preprint arXiv:2407.11770*, 2024. doi: 10.48550/arXiv.2407.11770. URL <https://arxiv.org/abs/2407.11770>. Extends BitNet b1.58 to smaller models; demonstrates that accuracy degradation increases significantly for models below 3B parameters, validating the need for capacity-restoration strategies like dual-expert design.
- N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017. URL <https://openreview.net/forum?id=B1ckMDqlg>. Foundational MoE architecture with sparsely-gated routing.
- Z. Wang et al. Bitnet.cpp: Efficient edge inference for ternary LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025)*, pages 457–470, 2025. URL <https://aclanthology.org/2025.acl-long.457.pdf>. Open-source inference engine achieving 2–4x CPU speedup for BitNet b1.58.

A Experiment Data Tables

A.1 Full Benchmark Comparison (from benchmark_comparison.csv)

Table 7: **Complete Benchmark Results.** Reproduced verbatim from `benchmark_comparison.csv`. The “Notes” column captures the Ultralytics checkpoint serialisation issue that caused the `Base5_YOLO_Conv` keys to be absent from the fine-tuned checkpoint (Step 4 format note).

Model	mAP50	mAP50-95	Params	Size (MB)	Notes
Baseline FP32 YOLOv8	0.6437	0.4788	2,408,932	5.29	Original pre-trained model.
Fine-tuned Standard YOLO (5-ep QAT)	0.0000	0.0000	2,408,932	10.13	Ultralytics serialised as standard YOLO; Base5 surgery confirmed during training but not preserved in checkpoint.
Dual-Expert Base-5 (Post-Surgery)	0.0000	0.0000	5,118,760	10.13	Theoretical $13.78\times$ compression confirmed. Expert_b random init degrades mAP; extended QAT needed.

A.2 Theoretical Efficiency Metrics (from final_experiment_report.json)

Table 8: **Theoretical Weight Storage Efficiency Metrics.** Computed for the 2,546,480 quantised convolutional weight parameters across the 113 replaced layers (per expert).

Metric	Value	Source
FP32 bits per weight	32.000 bits	IEEE 754 standard
Ternary bits per expert	1.585 bits ($\log_2 3$)	Information theory
Base-5 combined bits	2.322 bits ($\log_2 5$)	Information theory
FP32 storage (113 layers)	9.714 MB	$2,546,480 \times 32/8 \times 10^6$
Ternary per expert	0.481 MB	$2,546,480 \times 1.585/8 \times 10^6$
Base-5 fused	0.705 MB	$2,546,480 \times 2.322/8 \times 10^6$
Compression (FP32 $\rightarrow$ B5)	13.78$\times$	9.714/0.705
Compression (FP32 $\rightarrow$ single ternary)	20.19 $\times$	9.714/0.481

A.3 Model Surgery Report

Table 9: **Layer Replacement Statistics (Step 2 Surgery).**

Statistic	Value	Notes
Total Conv2d layers (YOLO26n)	126	Full backbone
Replaced with Base5_YOLO_Conv	113	89.7% coverage
Preserved in FP32	13	High-MSE + narrow-channel
MSE exclusion threshold	0.00758	Top 10% by MSE
Excluded by MSE criterion	13	
Excluded by out_channels==4	6	Detection head outputs
Combined exclusions	13	Union criterion
Forward pass validation	PASSED	<code>forward_pass_ok=True</code>
Modified model size	19.89 MB	Two FP32 experts in memory
Preserved layers (key)	<code>model.0.conv</code> (input layer), <code>model.2.cv1.conv</code> (early backbone), 11 detection head output projections	

A.4 QAT Configuration

Table 10: **Step 3 QAT Training Configuration.** Note the absence of FP32 warmup and gradient clipping—both identified as critical missing components in the failure analysis.

Parameter	Value	Status
Epochs	5	Insufficient for stable QAT
Image size	640×640	Standard
Batch size	4	CPU-limited
Device	CPU	No GPU acceleration
AMP	False	Full precision
Optimiser	AdamW	(not MuSGD—different from published YOLO26)
FP32 Warmup	None	MISSING: root cause #1
Gradient clipping	None	MISSING: root cause #2
Quantisation activation	Epoch 1	MISSING: should be epoch 6+

B ResiBit-YOLO Implementation Guide

B.1 Converting YOLO26n to ResiBit-YOLO

The following function traverses a YOLO26n model and replaces all eligible Conv2d layers with ResiBitConv blocks, using the same sensitivity-aware exclusion strategy as the Dual-Expert Base-5 experiment.

Listing 2: Model surgery: converting YOLO26n to ResiBit-YOLO.

```

1 import torch
2 import torch.nn as nn
3 from models.resibit_conv import ResiBitConv
4 from results.quantization_sensitivity import load_sensitivity_csv
5
6 def compute_quant_mse(layer: nn.Conv2d) -> float:
7     """Estimate ternary quantisation MSE for a Conv2d layer."""
8     W = layer.weight.data
9     alpha = W.abs().mean()
10    W_q = (W / alpha).round().clamp(-1, 1) * alpha
11    return ((W - W_q) ** 2).mean().item()
12
13 def replace_with_resibit(
14     model: nn.Module,
15     mse_threshold: float = 0.00758,
16     min_out_channels: int = 8) -> nn.Module:
17     """
18     Traverse 'model' and replace eligible Conv2d layers with
19     ResiBitConv.
20
21     Exclusion criteria (keep FP32):
22     - Quantisation MSE above 'mse_threshold'
23     - Output channels below 'min_out_channels'
24     - First convolutional layer (model.0.conv)
25     """
26     replaced, preserved = 0, 0
27     first_conv = True
28
29     for name, module in model.named_modules():
30         if not isinstance(module, nn.Conv2d):
31             continue
32
33         # Always preserve the first conv layer
34         if first_conv:
35             first_conv = False
36             preserved += 1
37             print(f" PRESERVED (first layer): {name}")
38             continue
39
40         mse = compute_quant_mse(module)
41         if mse > mse_threshold or module.out_channels <
42             min_out_channels:
43             preserved += 1
44             print(f" PRESERVED (MSE={mse:.5f}): {name}")
45             continue
46
47         # Replace with ResiBitConv
48         resibit = ResiBitConv(
49             in_channels=module.in_channels,
50             out_channels=module.out_channels,
51             kernel_size=module.kernel_size[0],

```

```

50         stride=module.stride[0],
51         padding=module.padding[0]
52     )
53     # Initialise expert A from original weights
54     with torch.no_grad():
55         resibit.w_a.copy_(module.weight)
56         resibit.w_b.copy_(module.weight * 0.1) # Perturbed init
57         resibit.w_r.copy_(module.weight)      # Full copy for
58         highway
59
60     # Perform the module replacement
61     parent, attr = name.rsplit('.', 1)
62     parent_module = dict(model.named_modules())[parent]
63     setattr(parent_module, attr, resibit)
64     replaced += 1
65     print(f"    REPLACED: {name}    (MSE={mse:.5f})")
66
67     print(f"\nSurgery complete: {replaced} replaced, {preserved}
68         preserved.")
69     return model

```

B.2 Training Loop Integration

The three-phase training loop for ResiBit-YOLO integrates with the Ultralytics YOLO26 training framework by overriding the `preprocess_batch` hook to apply per-phase quantisation scheduling.

Listing 3: Three-phase QAT training loop with gradient anchor activation.

```

1  class ResiBitTrainer(DetectionTrainer):
2      """Ultralytics trainer extension for ResiBit three-phase QAT."""
3
4      QAT_WARMUP_EPOCHS = 10    # Phase 1: no quantisation
5      QAT_ANNEAL_EPOCHS = 15    # Phase 2: gradual ternary warmup
6      QAT_FULL_EPOCHS = 300     # Phase 3: full QAT
7
8      def __init__(self, *args, **kwargs):
9          super().__init__(*args, **kwargs)
10         self._qat_phase = 1
11         self._anneal_step = 0
12
13     def preprocess_batch(self, batch):
14         epoch = self.epoch
15
16         # --- Phase 1: FP32 Warmup ---
17         if epoch <= self.QAT_WARMUP_EPOCHS:
18             for m in self.model.modules():
19                 if isinstance(m, ResiBitConv):
20                     m.quant_enabled = False
21             if epoch == 1:
22                 self.console.info("[ResiBit] Phase 1: FP32 warmup
23                     active.")
24
25         # --- Phase 2: Freeze INT8 highway, start ternary anneal ---
26         elif epoch <= self.QAT_WARMUP_EPOCHS + self.QAT_ANNEAL_EPOCHS:
27             frac = (epoch - self.QAT_WARMUP_EPOCHS) / self.
28                 QAT_ANNEAL_EPOCHS
29             anneal_scale = 3.0 * (1.0 - frac) + 1.0 * frac # 3->1
30             for m in self.model.modules():

```

```

29         if isinstance(m, ResiBitConv):
30             m.quant_enabled = True
31             m.anneal_scale = anneal_scale
32             # Freeze INT8 highway gradient
33             m.w_r.requires_grad_(False)
34         if epoch == self.QAT_WARMUP_EPOCHS + 1:
35             self.console.info(
36                 "[ResiBit] Phase 2: INT8 highway frozen, "
37                 "ternary warmup active.")
38
39         # --- Phase 3: Full ResiBit QAT ---
40         else:
41             for m in self.model.modules():
42                 if isinstance(m, ResiBitConv):
43                     m.quant_enabled = True
44                     m.anneal_scale = 1.0
45                     m.w_r.requires_grad_(True)
46             if epoch == self.QAT_WARMUP_EPOCHS + self.
47                 QAT_ANNEAL_EPOCHS + 1:
48                 self.console.info("[ResiBit] Phase 3: Full QAT active
49                                     .")
50
51         return super().preprocess_batch(batch)
52
53     def loss_additional(self):
54         """Collect orthogonal regularisation terms from all ResiBit
55         layers."""
56         orth_loss = torch.tensor(0.0, device=self.device)
57         for m in self.model.modules():
58             if isinstance(m, ResiBitConv):
59                 orth_loss = orth_loss + m.orthogonal_loss()
60         return orth_loss

```

B.3 Deployment: Fusing to ResiBit-Inference Format

After Phase 3 training converges, the dual ternary experts are fused into the Base-5 representation, producing a two-stream inference model (Base-5 + INT8) that requires no training-time infrastructure at runtime.

Listing 4: Post-training fusion of dual ternary experts to Base-5 format.

```

1  def fuse_resibit_for_inference(model: nn.Module) -> nn.Module:
2      """
3      Convert all ResiBitConv layers to inference format:
4      - Fuse w_a + w_b into Base-5 weight (INT8, {-2..2})
5      - Retain w_r as INT8 residual stream
6      - Replace ResiBitConv with ResiBitInferConv
7      """
8      for name, module in model.named_modules():
9          if not isinstance(module, ResiBitConv):
10              continue
11
12         # Base-5 fusion
13         w_b5 = module.get_base5_fused() # INT8 tensor, values
14             {-2, -1, 0, +1, +2}
15
16         # INT8 quantise residual highway
17         w_r_scale = module.w_r.abs().max() / 127.0

```



```
17     w_r_int8 = (module.w_r / w_r_scale).round().clamp(-128, 127).  
18         to(torch.int8)  
19  
20     # Build inference module  
21     infer_module = ResiBitInferConv(  
22         w_b5=w_b5,  
23         w_r_int8=w_r_int8,  
24         w_r_scale=w_r_scale,  
25         mix_weights=module._mix_weights().detach(),  
26         bn_weight=module.bn.weight.detach(),  
27         bn_bias=module.bn.bias.detach(),  
28         bn_mean=module.bn.running_mean.detach(),  
29         bn_var=module.bn.running_var.detach(),  
30     )  
31  
32     # Replace in model  
33     parent, attr = name.rsplit('.', 1)  
34     setattr(dict(model.named_modules())[parent], attr,  
35         infer_module)  
36     print(f"Fused: {name} B5 shape={w_b5.shape}, unique={w_b5.  
37         unique().tolist()}")  
38  
39     return model
```